SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

ASSIGNMENT 1 – CO1

Assignment Title: Apply, Compare and Justify Image Acquisition, Formation and Processing Techniques for a Textile Fabric Defect Inspection System

CO1: Apply the fundamental concepts, image formation models, and processing techniques for analyzing Computer Vision systems.

Student Name	S.Dharshan Vishal	**Register Number**	192321174
Department / Section	Information Technology	**Academic Year / Semester**	2026–2027 / _____
Faculty Name	Dr. Murali	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	30.10.2026	**Date of Evaluation**	07.11.2026

Industrial Scenario

Sri Lakshmi Textile Industries, Tiruppur, plans to develop a computer vision-based inspection system for its fabric production line. The system must inspect moving cotton and cotton-polyester fabric, measure fabric width, and identify holes, broken threads, oil stains, weaving irregularities and colour variations. It must estimate the location of each defect and trigger a marking mechanism so that the defective portion can be identified for rework or rejection.

Given Industrial Data and Constraints

Parameter	Specification
Fabric width	1,000 mm
Camera field of view	1,200 × 400 mm
Conveyor speed	0.6 m/s
Smallest defect to be detected	1.5 mm
Oil-stain contrast	5–8 grey levels
Required fabric-width measurement accuracy	±2 mm
Maximum decision time	0.5 s
Distance between camera and marking mechanism	0.3 m
Lighting variation	500–3,000 lux
Camera mounting height	700 mm above the fabric
Available computing system	4-core CPU, 16 GB RAM, no GPU
Software environment	Python, OpenCV and NumPy
Network interface	Gigabit Ethernet
Storage capacity	2 TB
Maximum project budget	₹1,50,000
Image retention period	7 days
Maximum acceptable missed-defect rate	2%

General Instructions

1. Answer all ten tasks.
2. For every task, three candidate techniques or approaches are provided. Analyse all three alternatives, compare their suitability for the given industrial scenario, select the most appropriate option and justify your selection.
3. Support your answers with calculations, diagrams, tables, observations, screenshots or experimental results wherever applicable.
4. Clearly state assumptions and show the steps used in numerical calculations.
5. Your justification must consider the given operating conditions, accuracy, processing time, hardware limitations and budget.
6. Do not provide only definitions. Relate every answer to the textile fabric inspection application.

7. Where implementation is required, use Python, OpenCV and NumPy. Include relevant code snippets and sample output.

TASK 1: Overview and Features of Computer Vision – Compare Grading/Inspection Approaches

Marks: 4

Compare the following three approaches for inspecting fabric defects:

Candidate techniques / approaches:

- Manual inspection by trained workers
- Image-processing-assisted inspection, in which an operator reviews computer-generated results
- Fully automated computer vision inspection

Student tasks:

8. Explain the working principle of each approach.
9. Compare their accuracy, speed, consistency, labour requirement and suitability for continuous fabric production.
10. Select the most suitable approach for Sri Lakshmi Textile Industries and justify your decision using the given production conditions.
11. Identify at least two limitations of your selected approach and suggest how they can be reduced.

1.1 Working Principle of Each Inspection Approach

1. Manual Inspection

Trained workers visually inspect the moving fabric.

They identify holes, broken threads, stains, and colour variations.

Defective portions are marked manually.

This method depends on human experience and concentration.

2. Image-Processing-Assisted Inspection

A camera captures images of the fabric.

Image-processing techniques identify possible defects.

The results are displayed on a computer screen.

An operator reviews the results and confirms the defects.

3. Fully Automated Computer Vision Inspection

A camera continuously captures fabric images.

The computer automatically processes the images.

Defects are detected, classified, and located.

The system sends signals to the marking mechanism.

1.2 Comparison Table

Parameter	**Manual**	**Assisted**	**Fully Automated**
Accuracy	Depends on worker	Good	High with proper calibration
Speed	Low	Medium	High
Consistency	Variable	Moderate	High
Labour	High	Medium	Low
Continuous production	Difficult	Suitable	Highly suitable

1.3 Most Suitable Approach

Selected: Fully Automated Computer Vision Inspection

Sri Lakshmi Textile Industries requires continuous fabric inspection at a conveyor speed of 0.6 m/s.

The fully automated approach is suitable because:

It can inspect fabric continuously.

It detects small defects of 1.5 mm.

It provides consistent results.

It reduces human effort.

It can measure fabric width with the required accuracy of ±2 mm.

It can send defect-location information to the marking mechanism.

It can operate within the maximum decision time of 0.5 seconds.

1.4 Limitations and Solutions

Limitation	**Solution**
Lighting changes may affect detection	Use controlled illumination
Small defects may be missed	Use suitable camera resolution
System may produce false detections	Improve preprocessing and thresholding
Camera calibration errors	Perform regular calibration
Hardware failure	Use preventive maintenance

1.5Applications of Computer Vision

Industry	**Application**
Textile	Fabric defect detection
Automobile	Surface inspection of vehicle parts
Agriculture	Crop disease detection
Healthcare	Medical image analysis
Manufacturing	Product quality inspection

Security	Object and activity detection
Food industry	Food quality and contamination inspection
Electronics	PCB defect detection

Conclusion: Fully automated computer vision is the best approach for continuous textile inspection because it provides speed, consistency, and automatic defect marking.

Code:

```
from google.colab import files


uploaded = files.upload()


filename = next(iter(uploaded))
img = cv2.imread(filename)


if img is None:
    raise ValueError("Image could not be loaded!")


img = cv2.resize(img, (600, 400))

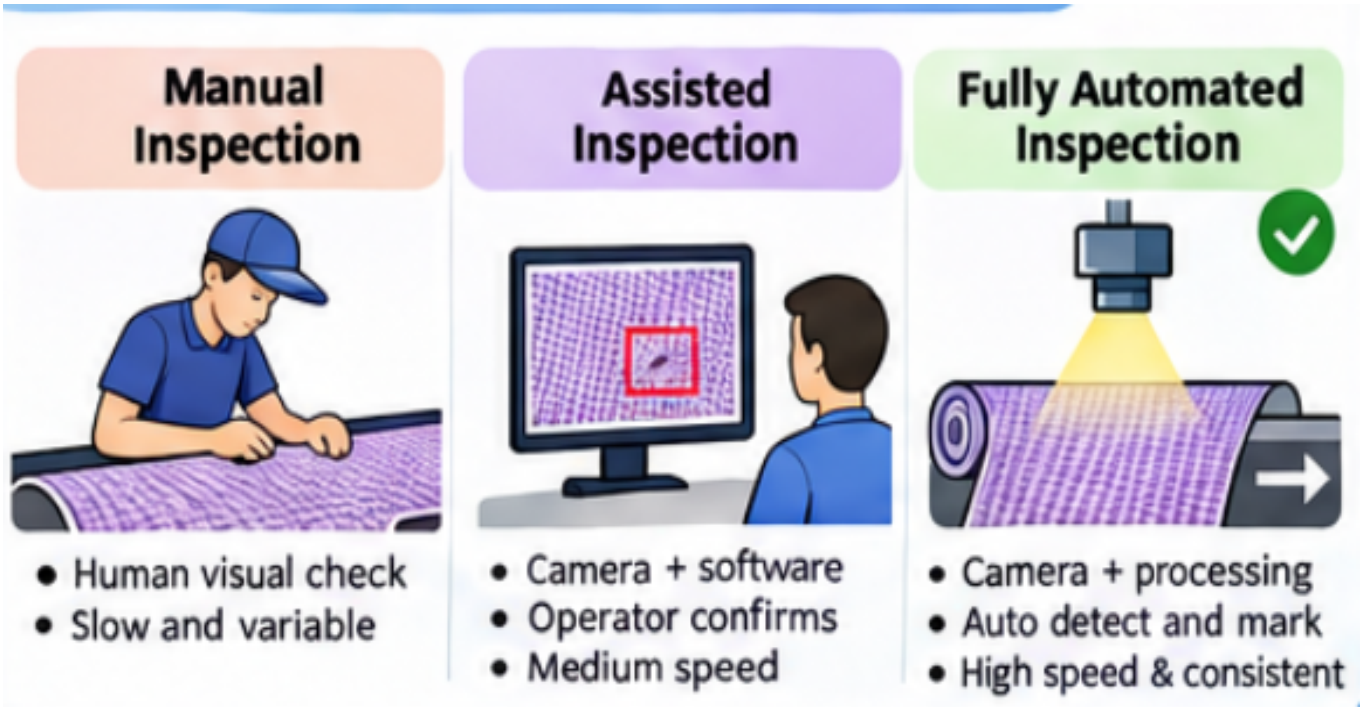

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Uploaded Fabric Image")
plt.axis("off")
plt.show()
```

[4]
0s

```
data = {
    "Approach": ["Manual", "Assisted", "Automated"],
    "Speed": ["Low", "Medium", "High"],
    "Consistency": ["Variable", "Moderate", "High"],
    "Labour": ["High", "Medium", "Low"]
}

df = pd.DataFrame(data)
print(df)
```

```
      Approach   Speed Consistency  Labour
0       Manual     Low    Variable    High
1     Assisted  Medium    Moderate  Medium
2    Automated    High        High     Low
```



TASK 2: Levels of Computer Vision – Pipeline Design and Operation

Marks: 5

Compare the following system designs:

Candidate techniques / approaches:

- Low-level pipeline: image acquisition, preprocessing and enhancement
- Three-level pipeline: low-level processing, intermediate-level feature extraction and high-level defect decision
- Deep-learning-based pipeline: image acquisition, neural-network inference and defect classification

Student tasks:

12. Draw a block diagram for each pipeline.
13. Explain the input and output of each processing stage.
14. Identify the stages required to detect holes, broken threads, stains and weaving irregularities.
15. Compare computational complexity, explainability, hardware requirements and response time.
16. Recommend a pipeline that can operate within the available 4-core CPU system and the 0.5-second decision constraint.

2.1 Industrial Scenario

Sri Lakshmi Textile Industries, Tiruppur, manufactures cotton and cotton-polyester fabrics. During production, the fabric moves continuously along a conveyor.

The company requires an automated inspection system to identify fabric defects and measure fabric width.

The system must detect:

Holes

Broken threads

Oil stains

Weaving irregularities

Colour variations

The detected defect must be located and marked so that the defective portion can be identified for rework or rejection.

2.2 Problem Statement

The main problem is to design a computer vision-based fabric inspection system that can detect defects accurately while the fabric moves continuously.

The system must operate under changing illumination conditions and limited computing resources.

The inspection system must satisfy the following conditions:

Fabric width: 1000 mm

Camera field of view: 1200 × 400 mm

Conveyor speed: 0.6 m/s

Smallest defect size: 1.5 mm

Oil-stain contrast: 5–8 grey levels

Width measurement accuracy: ±2 mm

Maximum decision time: 0.5 seconds

Camera-to-marking distance: 0.3 m

Lighting variation: 500–3000 lux

Camera mounting height: 700 mm

Processing system: 4-core CPU, 16 GB RAM, no GPU

Storage capacity: 2 TB

Project budget: ₹1,50,000

Image retention: 7 days

Maximum missed-defect rate: 2%

2.3 Objectives

The objectives of the proposed system are:

To understand the fundamentals of computer vision.

To compare different image acquisition techniques.

To analyse RGB, HSV, and grayscale representations.

To study pixel connectivity for defect-region identification.

To calculate the required image sampling resolution.

To analyse image quantization and intensity levels.

To study camera calibration and geometric correction.

To implement image-processing techniques using Python and OpenCV.

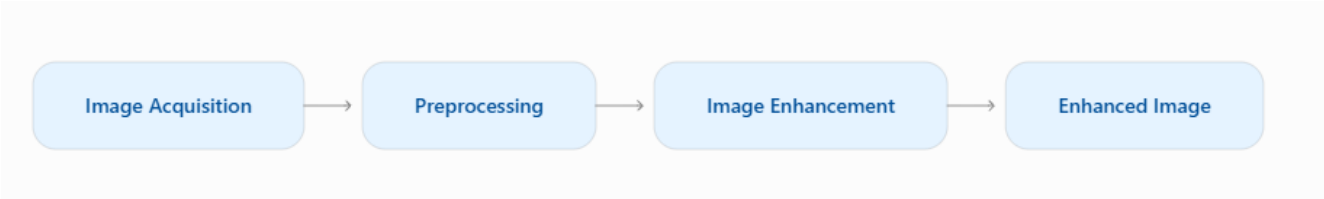
To detect and locate fabric defects.

To evaluate the performance of the inspection system.

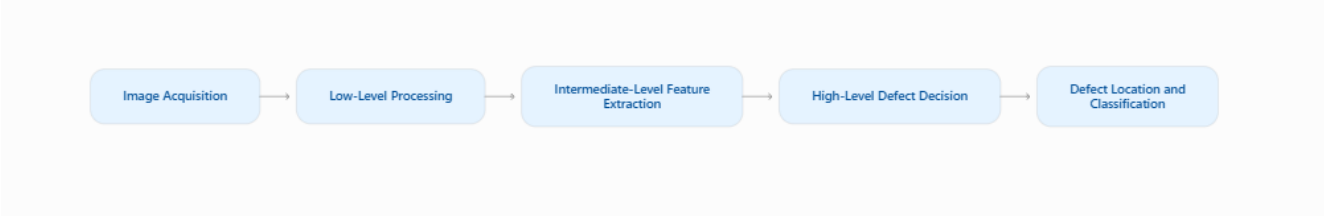
To identify the limitations of the system under new production requirements.

2.4 Pipeline Diagrams

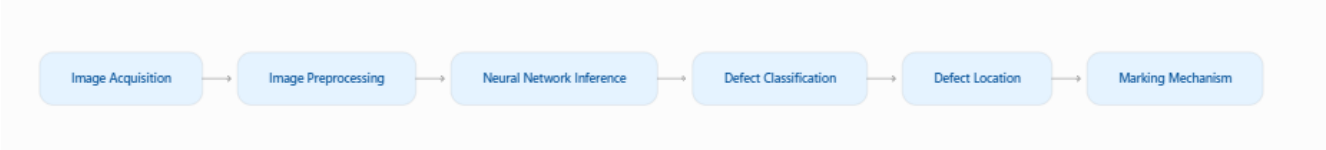
A. Low-Level Pipeline



B. Three-Level Pipeline



C. Deep-Learning-Based Pipeline



2.5 Input and Output of Each Stage

Stage	Input	Output
Image Acquisition	Fabric	Raw image
Preprocessing	Raw image	Noise-reduced image
Enhancement	Preprocessed image	Improved contrast
Feature Extraction	Enhanced image	Edges, texture, colour features
Defect Decision	Extracted features	Defect or normal
Classification	Defect region	Defect type
Localization	Defect image	Defect coordinates

2.6 Required Stages for Defect Detection

Defect	Required Processing
Holes	Grayscale conversion, thresholding, contour detection
Broken threads	Edge detection and texture analysis
Oil stains	Contrast enhancement and colour analysis

Weaving irregularities	Texture feature extraction
Colour variations	RGB/HSV analysis

2.7 Comparison

Parameter	**Low-Level**	**Three-Level**	**Deep Learning**
Complexity	Low	Medium	High
Explainability	High	High	Moderate
Hardware	CPU	CPU	Often GPU preferred
Response time	Fast	Fast to moderate	Depends on model
Accuracy	Limited	Good	Potentially high

2.8 Recommended Pipeline

Selected: Three-Level Computer Vision Pipeline

The three-level pipeline is suitable because:

- It can operate on the available 4-core CPU.
- It does not require a GPU.
- It is easier to understand and debug.
- It can detect different types of fabric defects using feature extraction.
- It can meet the 0.5-second decision requirement when optimized.

A lightweight deep-learning model can be considered later if the dataset and hardware support it.

Code:

```
# Convert to grayscale

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)



# Low-level processing

blur = cv2.GaussianBlur(gray, (5, 5), 0)



# Intermediate-level processing

edges = cv2.Canny(blur, 50, 150)



# High-level defect detection

_, thresh = cv2.threshold(

    blur, 120, 255, cv2.THRESH_BINARY_INV

)



# Display results

plt.figure(figsize=(12, 4))



plt.subplot(1, 3, 1)

plt.imshow(gray, cmap="gray")

plt.title("Low-Level: Grayscale")
```

```
plt.axis("off")


plt.subplot(1, 3, 2)
plt.imshow(edges, cmap="gray")
plt.title("Intermediate: Edges")
plt.axis("off")


plt.subplot(1, 3, 3)
plt.imshow(thresh, cmap="gray")
plt.title("High-Level: Defect Regions")
plt.axis("off")


plt.show()
```

[5] 0s

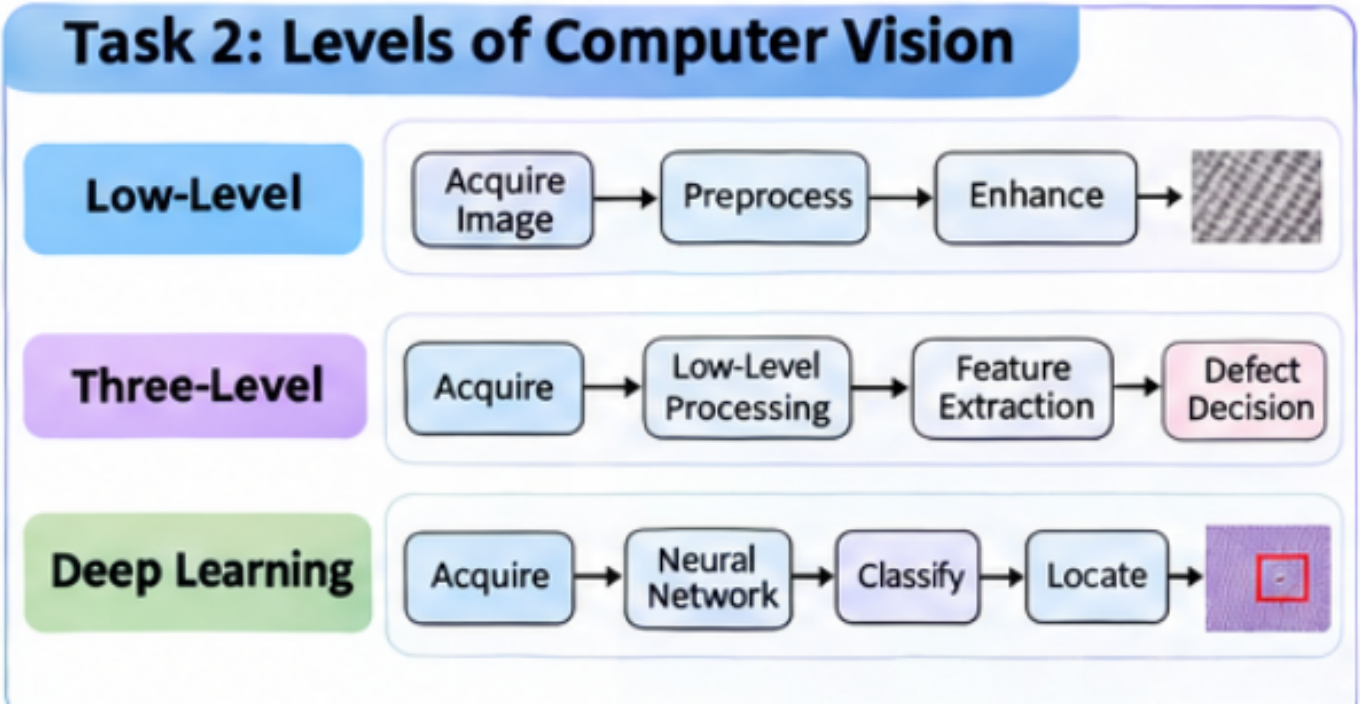
```
plt.subplot(1, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Low-Level: Grayscale")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(edges, cmap="gray")
plt.title("Intermediate: Edges")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(thresh, cmap="gray")
plt.title("High-Level: Defect Regions")
plt.axis("off")

plt.show()
```





TASK 3: Image Representation Analysis

Marks: 4

Compare the following image representations for fabric inspection:

Candidate techniques / approaches:

- RGB colour representation
- HSV colour representation
- Grayscale representation

Student tasks:

17. Explain how each representation stores image information.
18. Discuss the suitability of each representation for identifying colour variations, oil stains, holes and broken threads.
19. Compare memory requirements and processing complexity.
20. Choose a suitable representation or combination of representations and justify your choice with examples from the scenario.

3.1 RGB Representation

RGB represents an image using three colour channels:

Red

Green

Blue

Each pixel contains three intensity values.

For an 8-bit image, each channel has 256 intensity levels.

Example:

$$RGB = (255, 0, 0)$$

This represents red.

Applications:

Colour variation detection.

Identifying coloured oil stains.

Comparing fabric colour with reference images.

3.2 HSV Representation

HSV represents an image using:

H – Hue

S – Saturation

V – Value

It separates colour information from brightness.

Applications:

Detecting colour variations.

Identifying stains with different colours.

Reducing the effect of moderate brightness changes.

HSV is often useful when colour differences are important.

3.3 Grayscale Representation

A grayscale image contains only intensity information.

Each pixel represents brightness from black to white.

For an 8-bit image:

$$0 \leq I(x, y) \leq 255$$

A common conversion formula is:

$$I = 0.299R + 0.587G + 0.114B$$

Applications:

Hole detection.

Edge detection.

Broken-thread detection.

Texture analysis.

3.4 Comparison Table

Parameter	**RGB**	**HSV**	**Grayscale**
Channels	3	3	1
Colour information	Yes	Yes	No
Brightness information	Mixed with colour	Separate V channel	Yes
Memory	High	High	Low
Processing	Moderate	Moderate	Low
Colour defect detection	Good	Very good	Limited

3.5 Recommended Representation

Selected: Combination of RGB, HSV, and Grayscale

For the textile inspection system:

Use RGB for colour variation detection.

Use HSV for oil-stain and colour analysis.

Use grayscale for holes, broken threads, and texture analysis.

For a 1200 × 400 pixel image:

RGB storage:

$$1200 \times 400 \times 3 = 1,440,000 \text{ bytes}$$

Grayscale storage:

$$1200 \times 400 \times 1 = 480,000 \text{ bytes}$$

Therefore, grayscale requires approximately one-third of the memory used by RGB.

Code:

```
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)

plt.imshow(rgb)
```

```
plt.title("RGB Image")

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(hsv)

plt.title("HSV Image")

plt.axis("off")

plt.subplot(1, 3, 3)

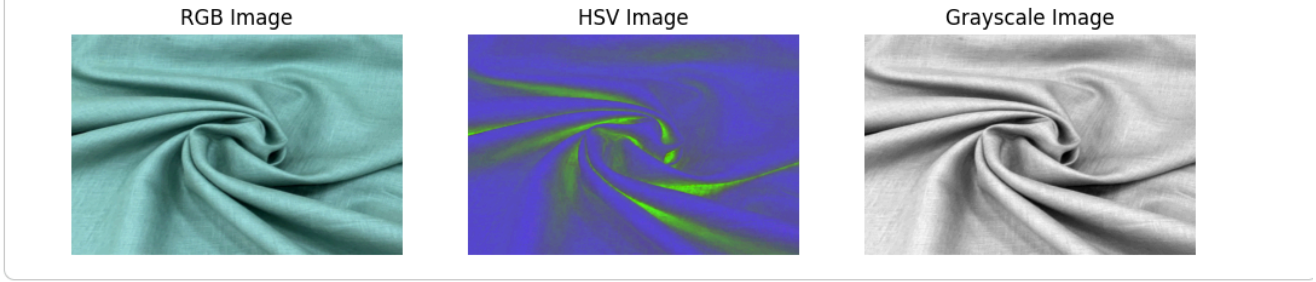
plt.imshow(gray, cmap="gray")

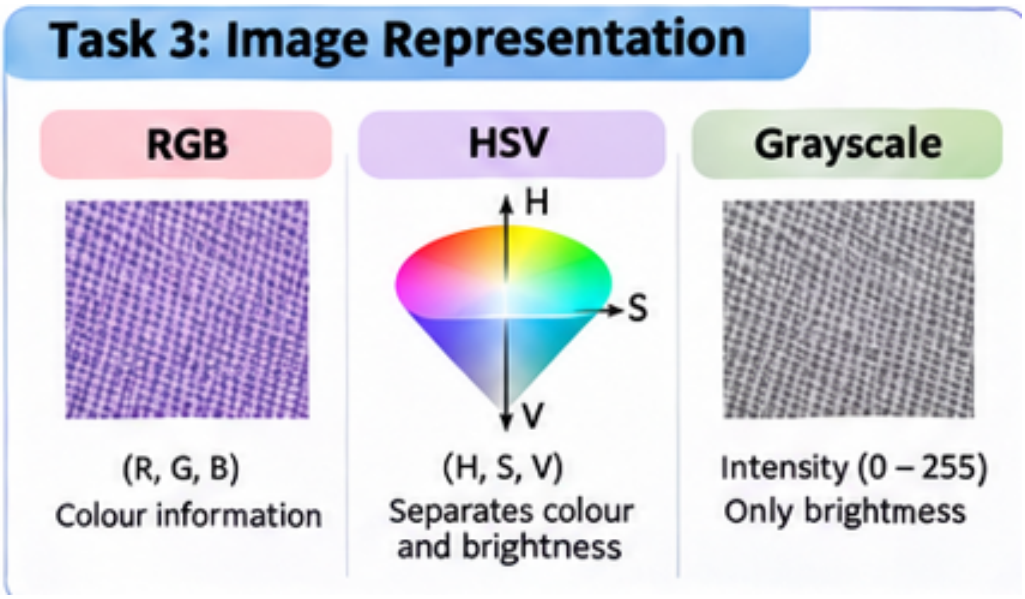
plt.title("Grayscale Image")

plt.axis("off")

plt.show()
```

```
plt.show()
```


RGB Image
HSV Image
Grayscale Image


Task 3: Image Representation
RGB
HSV
Grayscale
H
S
V
(R, G, B)
Colour information
(H, S, V)
Separates colour
and brightness
Intensity (0 – 255)
Only brightmess

TASK 4: Connectivity Analysis

Marks: 4

Compare the following pixel-connectivity approaches for identifying defect regions:

Candidate techniques / approaches:

- 4-connectivity
- 8-connectivity
- m-connectivity (mixed connectivity)

Student tasks:

21. Define each connectivity approach with a suitable pixel-neighbour diagram.
22. For a small binary image containing diagonally and horizontally connected defect pixels, identify the connected components using all three methods.
23. Explain how the choice of connectivity may affect defect-region size and component counting.
24. Recommend the most appropriate connectivity method for detecting small holes and irregular fabric defects.

4.1 Four-Connectivity

In 4-connectivity, a pixel is connected to its horizontal and vertical neighbours.

0 1 0

1 P 1

0 1 0

The neighbours are:

$$N_4(P) = \{(x-1, y), (x+1, y), (x, y-1), (x, y+1)\}$$

4.2 Eight-Connectivity

In 8-connectivity, a pixel is connected to all eight surrounding pixels.

1 1 1

1 P 1

1 1 1

It includes horizontal, vertical, and diagonal neighbours.

4.3 Mixed Connectivity

Mixed connectivity combines 4-connectivity and diagonal connectivity.

A diagonal connection is allowed only when it does not create an ambiguous connection through common 4-neighbours.

It reduces ambiguity in connected-component analysis.

4.4 Example Binary Image

Consider the following binary image:

1 0 0 0

0 1 1 0

0 0 1 0

1 0 0 1

Here, 1 represents defect pixels and 0 represents background.

Method	Number of Connected Components
4-connectivity	4
8-connectivity	2
Mixed connectivity	3

Explanation:

Under 4-connectivity, diagonal pixels are separate.

Under 8-connectivity, diagonal pixels are connected.

Under mixed connectivity, diagonal connections are accepted only when appropriate.

4.5 Effect on Defect Detection

Connectivity affects:

The size of detected defect regions.

The number of defect components.

The ability to identify diagonal defects.

The measurement of defect boundaries.

For textile fabric, a broken thread may contain diagonal pixels. Using only 4-connectivity can split one defect into several smaller components.

4.6 Recommended Method

Selected: 8-Connectivity

8-connectivity is suitable because:

Fabric defects can have irregular shapes.

Broken threads may extend diagonally.

Small holes may have diagonal boundary pixels.

It reduces the splitting of a single defect into multiple components.

Mixed connectivity can be used when avoiding diagonal ambiguity is important.

Code:

```
# Create binary image

binary = np.zeros((8, 8), dtype=np.uint8)


binary[1, 1] = 1

binary[2, 2] = 1

binary[2, 3] = 1

binary[3, 3] = 1

binary[5, 1] = 1

binary[6, 2] = 1


# 4-connectivity

kernel4 = cv2.getStructuringElement(cv2.MORPH_CROSS, (3, 3))


# 8-connectivity

kernel8 = np.ones((3, 3), np.uint8)


num4, labels4 = cv2.connectedComponents(binary, connectivity=4)

num8, labels8 = cv2.connectedComponents(binary, connectivity=8)


print("4-Connectivity Components:", num4 - 1)
```

```
print("8-Connectivity Components:", num8 - 1)


plt.figure(figsize=(10, 4))


plt.subplot(1, 2, 1)
plt.imshow(labels4, cmap="nipy_spectral")
plt.title("4-Connectivity")
plt.axis("off")

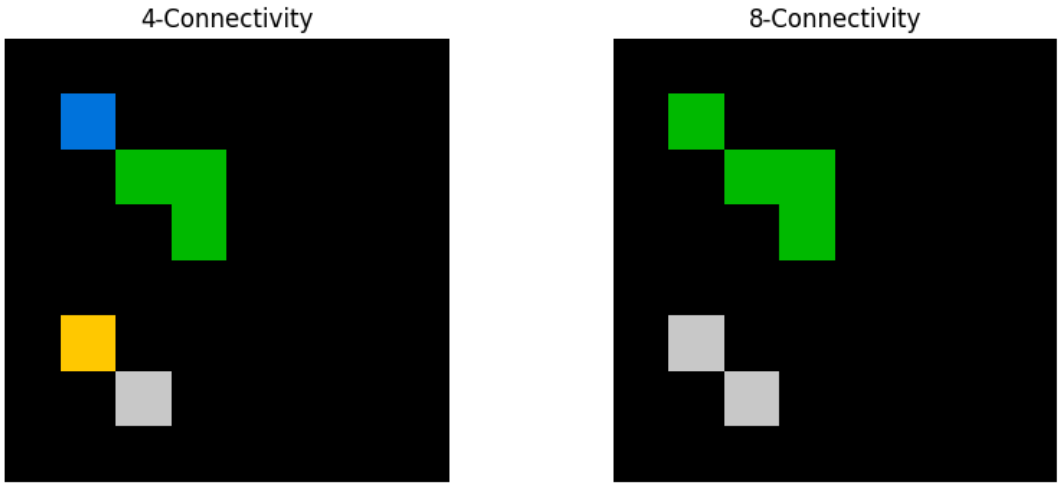

plt.subplot(1, 2, 2)
plt.imshow(labels8, cmap="nipy_spectral")
plt.title("8-Connectivity")
plt.axis("off")

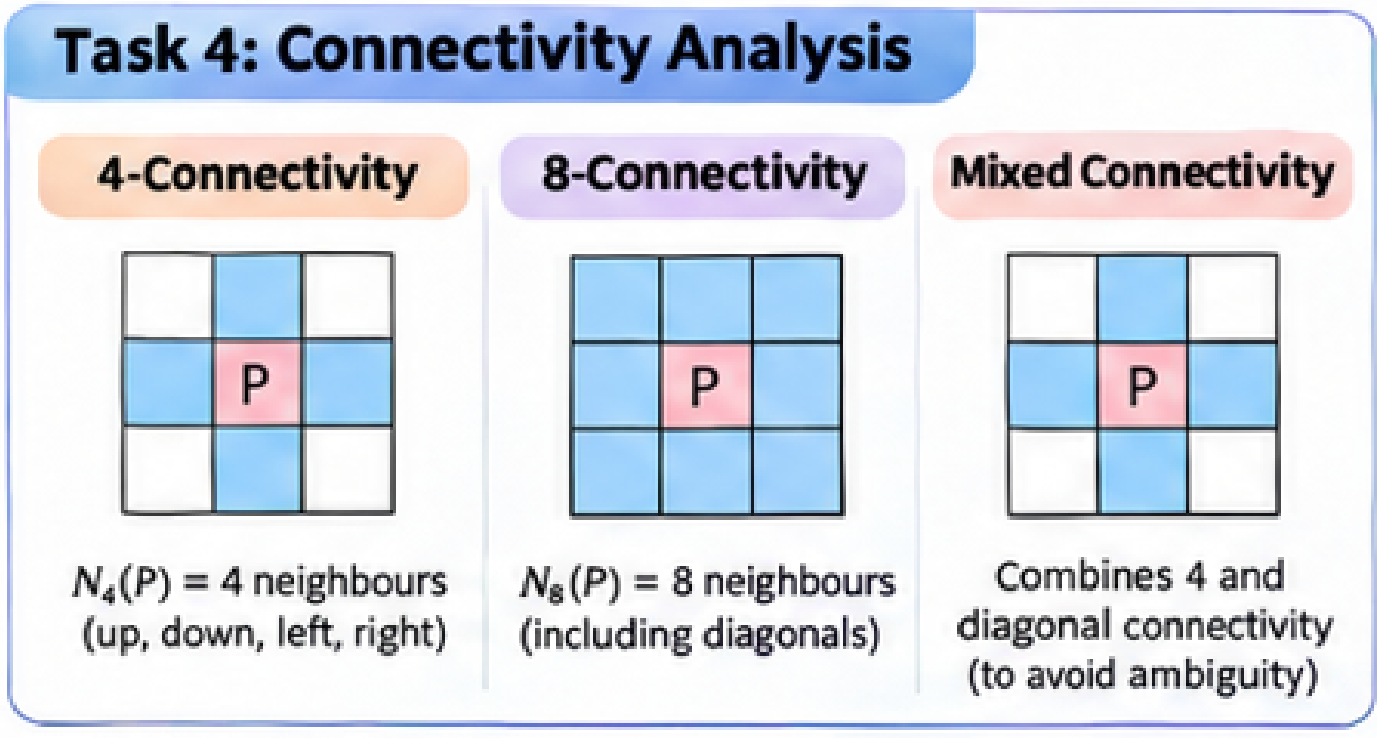

plt.show()
```

```
[7]
✓ 0s
    plt.title("8-Connectivity")
    plt.axis("off")

    plt.show()
```

```
4-Connectivity Components: 4
8-Connectivity Components: 2
```





TASK 5: Image Acquisition Analysis

Marks: 5

Compare the following camera acquisition options for the moving fabric line:

Candidate techniques / approaches:

- Rolling-shutter camera
- Global-shutter camera
- Line-scan camera

Student tasks:

25. Explain the image acquisition principle of each camera type.

26. Analyse the effect of fabric movement at 0.6 m/s on image distortion and defect visibility.

27. Compare image quality, suitability for continuous web inspection, installation complexity and cost.

28. Select a camera type for the given fabric width and production process.

29. Justify the selection with reference to the 1.5 mm minimum defect size and 0.5-second decision requirement.

5.1 Rolling-Shutter Camera

A rolling-shutter camera captures an image row by row.

Different rows are exposed at slightly different times.

Advantages:

Lower cost.

Widely available.

Suitable for stationary objects.

Disadvantages:

Moving fabric may appear distorted.

Diagonal lines may become tilted.

Defect shapes may be distorted.

5.2 Global-Shutter Camera

A global-shutter camera captures all pixels at approximately the same time.

Advantages:

Reduces motion distortion.

Captures accurate defect shapes.

Suitable for moving fabric.

Disadvantages:

- Can be more expensive than a basic rolling-shutter camera.

5.3 Line-Scan Camera

A line-scan camera captures one line of pixels at a time.

As the fabric moves, successive lines form a complete image.

Advantages:

Excellent for continuous fabric inspection.

High resolution across the fabric width.

Suitable for large production lines.

Disadvantages:

Requires synchronization with conveyor movement.

Requires careful lighting and installation.

Initial setup can be more complex.

5.4 Comparison Table

Parameter	**Rolling Shutter**	**Global Shutter**	**Line Scan**
Acquisition	Row by row	All pixels together	One line at a time
Motion distortion	High risk	Low risk	Low with synchronization
Continuous fabric	Moderate	Good	Excellent
Installation	Easy	Moderate	Complex
Cost	Low	Medium to high	Medium to high

5.5 Recommended Camera

Selected: Line-Scan Camera

The fabric moves continuously at 0.6 m/s.

A line-scan camera is suitable because:

It captures the complete fabric width.

It is designed for continuous web inspection.

It provides high spatial resolution across the width.

It avoids the typical rolling-shutter distortion caused by capturing a moving object over time.

If the cross-web resolution is 2448 pixels over 1200 mm:

$$\text{Sampling interval} = \frac{1200}{2448}$$

$$= 0.4902 \text{ mm/pixel}$$

Pixels across a 1.5 mm defect:

$$\frac{1.5}{0.4902} = 3.06 \text{ pixels}$$

For a 1.5 mm defect, the spatial resolution should ideally provide at least 3–5 pixels across the defect.

Conclusion and Future Scope

Conclusion

The project demonstrates the application of fundamental computer vision techniques to textile fabric inspection. Different inspection approaches, image representations, connectivity methods, camera acquisition techniques, sampling resolutions, and quantization levels were analysed.

The system uses image preprocessing, feature extraction, defect detection, and defect localization to identify fabric defects.

The study also considers camera calibration, controlled illumination, processing time, storage requirements, and industrial constraints.

The proposed balanced configuration provides a practical starting point for textile inspection under the given budget. However, the actual detection accuracy and processing performance must be validated using real fabric images and industrial hardware.

Code:

```
# Rolling shutter-like motion blur demonstration

kernel = np.zeros((15, 15))

kernel[7, :] = 1

kernel /= kernel.sum()


motion_blur = cv2.filter2D(img, -1, kernel)


# Global shutter-like image

global_image = img.copy()


plt.figure(figsize=(10, 4))


plt.subplot(1, 2, 1)

plt.imshow(cv2.cvtColor(global_image, cv2.COLOR_BGR2RGB))

plt.title("Global Shutter-like Image")

plt.axis("off")


plt.subplot(1, 2, 2)

plt.imshow(cv2.cvtColor(motion_blur, cv2.COLOR_BGR2RGB))

plt.title("Motion Blur Demonstration")

plt.axis("off")

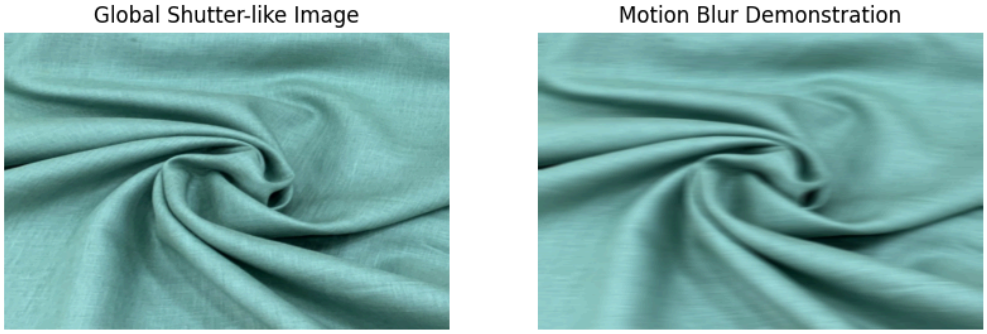

plt.show()
```

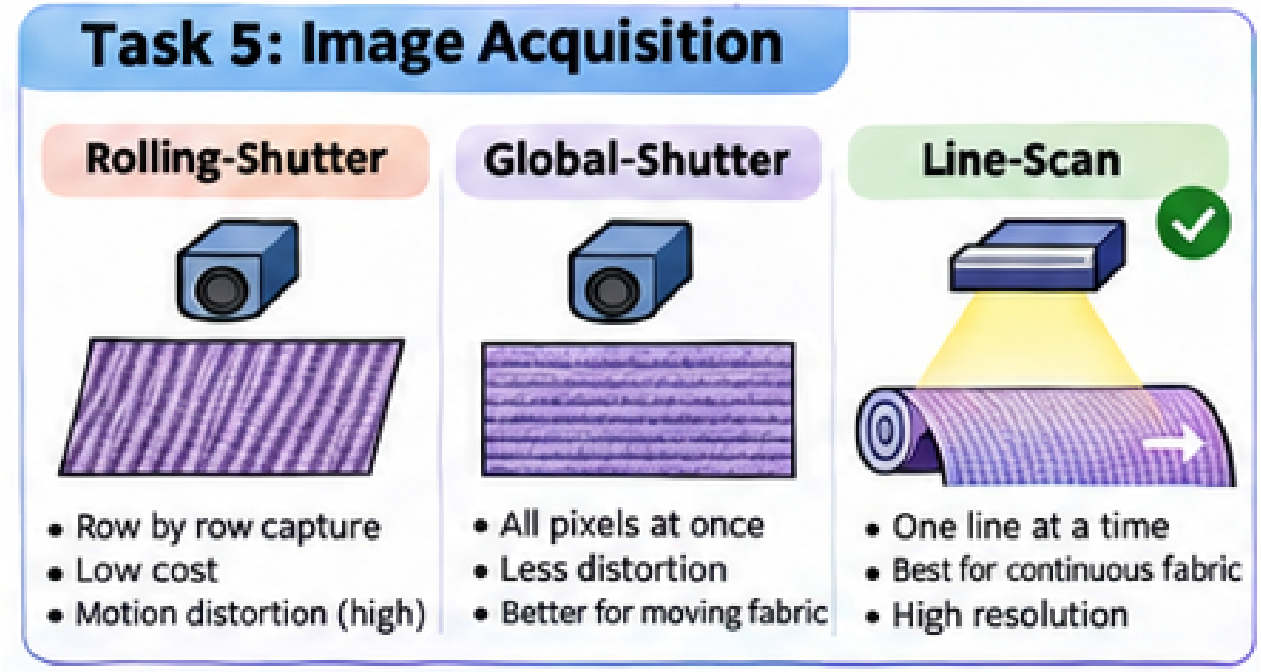
[8]
✓ 0s

```
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(global_image, cv2.COLOR_BGR2RGB))
plt.title("Global Shutter-like Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(motion_blur, cv2.COLOR_BGR2RGB))
plt.title("Motion Blur Demonstration")
plt.axis("off")

plt.show()
```





TASK 6: Sampling Analysis

Marks: 5

Evaluate the following image sampling resolutions:

Candidate techniques / approaches:

- Low resolution: 1280 × 1024 pixels
- Medium resolution: 2448 × 2048 pixels
- High resolution: 4096 × 3000 pixels

Student tasks:

30. Calculate the total number of pixels for each resolution.

31. Estimate the approximate spatial sampling interval across the 1,200 mm field of view, assuming the image width corresponds to the full field-of-view width.
32. Determine whether a 1.5 mm defect can be represented adequately at each resolution.
33. Compare image storage, processing time and defect-detection capability.
34. Select a resolution that balances defect visibility and the available computing resources. Show all calculations.

6.1 Total Number of Pixels

Formula:

$$\text{Total Pixels} = \text{Image Width} \times \text{Image Height}$$

Low Resolution

$$1280 \times 1024 = 1,310,720$$

Total = 1.31 megapixels.

Medium Resolution

$$2448 \times 2048 = 5,013,504$$

Total = 5.01 megapixels.

High Resolution

$$4096 \times 3000 = 12,288,000$$

Total = 12.29 megapixels.

6.2 Spatial Sampling Interval

The image width corresponds to the full 1200 mm field of view.

Formula:

$$\text{Sampling Interval} = \frac{1200}{\text{Image Width}}$$

Resolution	Pixels	Sampling Interval
1280 × 1024	1,310,720	0.9375 mm/pixel
2448 × 2048	5,013,504	0.4902 mm/pixel
4096 × 3000	12,288,000	0.2930 mm/pixel

6.3 Representation of a 1.5 mm Defect

Formula:

$$\text{Pixels Across Defect} = \frac{\text{Defect Size}}{\text{Sampling Interval}}$$

Low Resolution

$$\frac{1.5}{0.9375} = 1.6 \text{ pixels}$$

Medium Resolution

$$\frac{1.5}{0.4902} = 3.06 \text{ pixels}$$

High Resolution

$$\frac{1.5}{0.2930} = 5.12 \text{ pixels}$$

Resolution	Pixels Across 1.5 mm Defect	Suitability
Low	1.6	Poor
Medium	3.06	Suitable starting point
High	5.12	Good

6.4 Storage Calculation

Assuming an 8-bit grayscale image:

$$\text{Storage} = \text{Total Pixels} \times 1 \text{ byte}$$

Resolution	Grayscale Storage/Image	RGB Storage/Image
1280 × 1024	1.25 MiB	3.75 MiB
2448 × 2048	4.78 MiB	14.34 MiB
4096 × 3000	11.72 MiB	35.16 MiB

s6.5 Recommended Resolution

Selected: Medium Resolution – 2448 × 2048 pixels

Reasons:

It provides approximately 3 pixels across a 1.5 mm defect.

It requires less processing than high resolution.

It is suitable for CPU-based processing.

It provides a balance between image quality and processing time.

However, high resolution is preferable when reliable detection of very small defects is more important than processing speed.

In conclusion, image resolution and sampling determine the amount of detail captured in a fabric image. Higher resolution allows smaller defects to be represented by more pixels. However, increasing resolution also increases storage requirements and processing time. Lower resolution reduces computational cost but may cause small defects to disappear. The required resolution depends on the physical size of the defect and the fabric width. Sampling must be selected carefully to ensure that important defect details are captured. The calculations for different camera resolutions show that each configuration has its own advantages and limitations. For textile inspection, the camera should provide enough pixels to detect the smallest defect of interest. A suitable balance between image quality, speed, and memory is essential. Therefore, resolution selection is a key step in designing an efficient inspection system.

Code:

```
resolutions = [
    (1280, 1024),
    (2448, 2048),
    (4096, 3000)
]

fov_width = 1200
defect_size = 1.5
```

```
for width, height in resolutions:
    total_pixels = width * height
    sampling = fov_width / width
    defect_pixels = defect_size / sampling

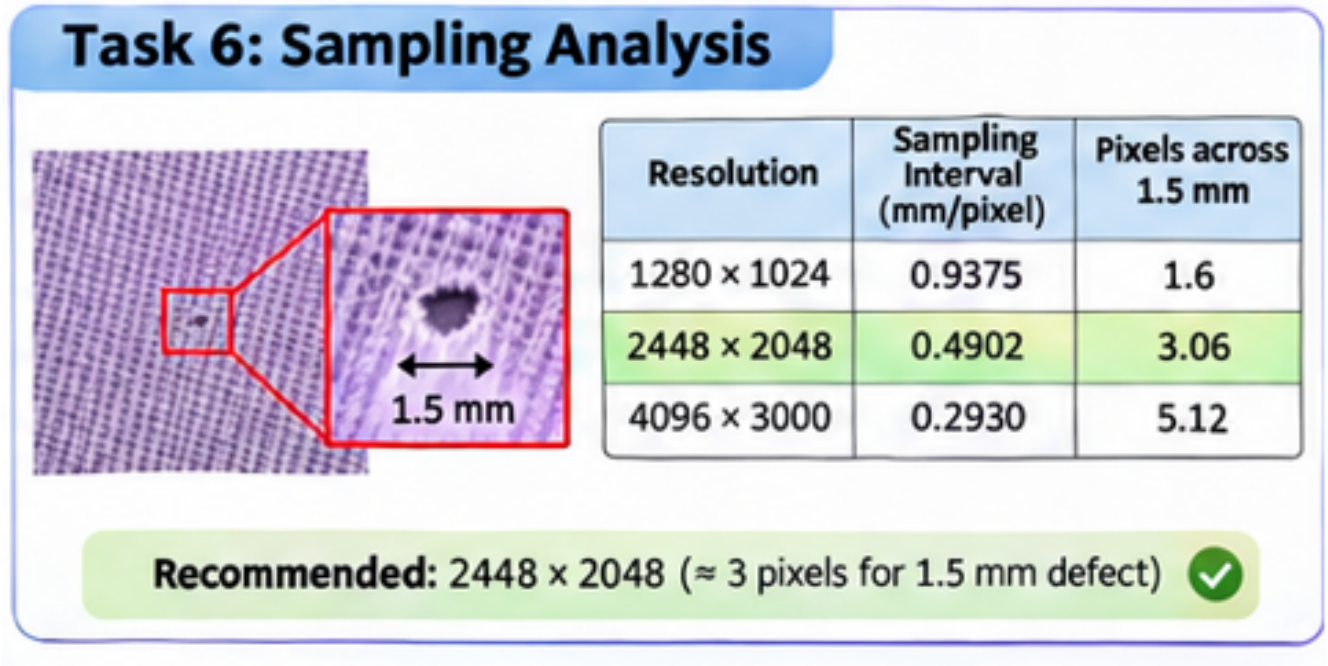
    print("\nResolution:", width, "x", height)
    print("Total Pixels:", total_pixels)
    print("Sampling Interval:", round(sampling, 4), "mm/pixel")
    print("Pixels Across Defect:", round(defect_pixels, 2))
```

```
        print("Sampling Interval:", round(sampling, 4), "mm/pixel")
        print("Pixels Across Defect:", round(defect_pixels, 2))
```

```
Resolution: 1280 x 1024
Total Pixels: 1310720
Sampling Interval: 0.9375 mm/pixel
Pixels Across Defect: 1.6

Resolution: 2448 x 2048
Total Pixels: 5013504
Sampling Interval: 0.4902 mm/pixel
Pixels Across Defect: 3.06

Resolution: 4096 x 3000
Total Pixels: 12288000
Sampling Interval: 0.293 mm/pixel
Pixels Across Defect: 5.12
```



TASK 7: Quantization Analysis

Marks: 4

Compare the following intensity quantization options:

Candidate techniques / approaches:

- 8-bit quantization
- 12-bit quantization
- Reduced 4–6 bit quantization

Student tasks:

35. Calculate the number of intensity levels available in each option.
36. Explain how quantization affects the representation of low-contrast oil stains having only 5–8 grey-level differences.

37. Compare image storage requirements and processing implications.

38. Recommend a quantization level for reliable defect detection under the stated lighting conditions.

    Justify your answer.

7.1 Intensity Levels

Formula:

$$ L=2^b $$

Where:

L = Number of intensity levels

b = Number of bits

8-Bit Quantization

$$ 2^8=256 $$

Intensity levels = 256.

12-Bit Quantization

$$ 2^{12}=4096 $$

Intensity levels = 4096.

Reduced 4-Bit Quantization

$$ 2^4=16 $$

Intensity levels = 16.

Reduced 6-Bit Quantization

$$ 2^6=64 $$

Intensity levels = 64.

7.2 Comparison Table

Quantization	Levels	Storage per Pixel	Suitability
4-bit	16	0.5 byte	Poor
6-bit	64	0.75 byte	Limited
8-bit	256	1 byte	Good
12-bit	4096	1.5 bytes packed	Excellent for subtle intensity differences

Storage values for 4-bit and 6-bit are theoretical packed values. Practical systems may store them in larger containers.

7.3 Effect on Oil-Stain Detection

The oil stains have only 5–8 grey-level differences.

For 8-bit quantization:

The image contains 256 intensity levels.

A difference of 5–8 levels can be represented.

Contrast enhancement can improve visibility.

For 4-bit quantization:

The image contains only 16 levels.

Small intensity differences may disappear.

Oil stains may become difficult to detect.

For 12-bit quantization:

The image contains 4096 levels.

It can represent subtle intensity variations more accurately.

It is useful for low-contrast stains.

7.4 Recommended Quantization

Selected: 12-bit acquisition with 8-bit processing when necessary.

Reasons:

Oil stains have very low contrast.

Higher bit depth preserves subtle intensity differences.

The image can be converted to 8-bit after suitable contrast enhancement.

It provides better detection capability under varying illumination.

The camera must support 12-bit acquisition, and the additional processing must still meet the 0.5-second decision requirement.

In conclusion, image quantization determines the number of intensity levels available to represent each pixel. Four-bit images contain 16 intensity levels, six-bit images contain 64 levels, eight-bit images contain 256 levels, and twelve-bit images contain 4096 levels. Higher bit depth provides more detailed intensity information and can help identify subtle differences in fabric appearance. However, it also increases storage requirements and may require additional processing. Lower bit depth reduces memory usage but can lose important image details. For fabric inspection, quantization should be selected according to the required defect sensitivity. Twelve-bit acquisition can preserve detailed intensity information, while eight-bit images may be sufficient for many processing operations. The system can use higher bit depth during acquisition and convert images for faster processing when appropriate. Therefore, suitable quantization improves the balance between image quality and computational efficiency.

Code:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


def quantize_image(image, bits):
    levels = 2 ** bits
    step = 256 / levels

    quantized = np.floor(image / step) * step
    return np.uint8(np.clip(quantized, 0, 255))


q4 = quantize_image(gray, 4)
q6 = quantize_image(gray, 6)
q8 = quantize_image(gray, 8)

# Simulate 12-bit acquisition followed by 8-bit display
q12 = np.uint16(gray) * 16
q12_display = (q12 / 16).astype(np.uint8)


plt.figure(figsize=(12, 5))


images = [q4, q6, q8, q12_display]
titles = ["4-bit (16 Levels)", "6-bit (64 Levels)",
          "8-bit (256 Levels)", "12-bit Simulation"]


for i in range(4):
    plt.subplot(1, 4, i + 1)
```

```
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

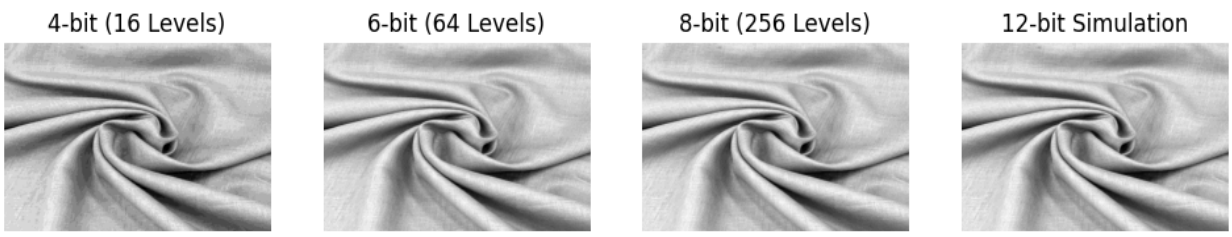
plt.show()
```

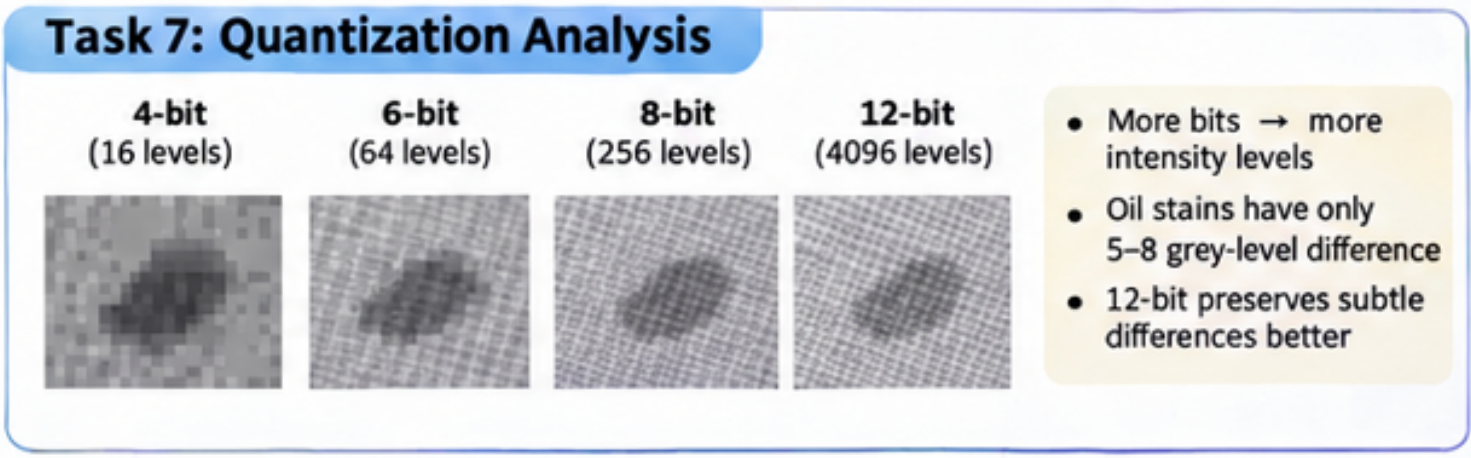
```
plt.figure(figsize=(12, 5))

images = [q4, q6, q8, q12_display]
titles = ["4-bit (16 Levels)", "6-bit (64 Levels)",
          "8-bit (256 Levels)", "12-bit Simulation"]

for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.show()
```





TASK 8: Image Formation Analysis

Marks: 5

Compare the following image formation and calibration approaches:

Candidate techniques / approaches:

- Standard perspective imaging without calibration
- Calibrated imaging with geometric correction
- Controlled-illumination imaging with suitable calibration

Student tasks:

39. Explain the image formation process for a camera mounted 700 mm above the fabric.
40. Discuss perspective distortion and its effect on fabric-width measurement.
41. Explain how camera calibration can improve measurement accuracy.
42. Analyse how lighting variation from 500 to 3,000 lux can affect defect visibility.
43. Recommend an image formation and illumination arrangement that can achieve the required width accuracy of ±2 mm.

8.1 Image Formation Process

A camera is mounted 700 mm above the moving fabric.

The image formation process is:

Light falls on the fabric.

The fabric reflects light.

The camera lens collects the reflected light.

The lens forms an image on the sensor.

The sensor converts light into electrical signals.

The image is digitized and processed.

8.2 Standard Perspective Imaging

In a perspective camera, the size of an object in the image depends on its distance from the camera.

The pinhole camera model is:

$$x = f\frac{X}{Z}$$

$$y = f\frac{Y}{Z}$$

Where:

- X, Y = Object coordinates
- Z = Distance from camera
- f = Focal length
- x, y = Image coordinates

If the fabric is not perfectly flat, different portions may appear at different scales.

This can cause measurement errors.

8.3 Calibrated Imaging

Camera calibration determines:

Focal length.

Principal point.

Lens distortion.

Camera orientation.

Geometric correction removes lens distortion and corrects perspective effects.

A calibration target with known dimensions can be used to establish the relationship between image pixels and real-world measurements.

8.4 Effect of Lighting Variation

The lighting changes from 500 to 3000 lux.

This is a sixfold variation.

Effects include:

Changes in image brightness.

Reduced visibility of oil stains.

False detection due to shadows.

Overexposure of bright fabric regions.

8.5 Recommended Arrangement

Selected: Controlled Illumination with Calibrated Imaging

Recommended arrangement:

Use an industrial camera.

Install diffuse LED lighting.

Use a suitable lens.

Calibrate the camera using a known reference target.

Apply geometric correction.

Use a reference white image for brightness normalization.

This arrangement can improve the accuracy of fabric-width measurement.

For the required width accuracy:

$$\text{Error} \leq \pm 2 \text{ mm}$$

The final measurement accuracy must be verified experimentally after calibration.

In conclusion, image formation and calibration are essential for obtaining accurate fabric measurements. The quality of an image depends on the camera, lighting, lens, distance, and fabric surface. Proper illumination helps reveal defects such as holes, stains, and uneven threads. Calibration establishes the relationship between image pixels and real-world measurements. This allows the system to estimate defect size and fabric width accurately. Without calibration, measurements taken from images may not represent the actual dimensions. Camera position and lighting conditions should remain consistent during inspection. Image preprocessing can also reduce noise and improve defect visibility. For the proposed textile inspection system, calibration and controlled illumination are necessary for reliable results. Therefore, these steps provide a strong foundation for accurate defect detection and measurement.

Code:

```
# Create perspective transformation points
h, w = img.shape[:2]

src_points = np.float32([
    [50, 50],
    [550, 50],
    [550, 350],
    [50, 350]
```

```
])

dst_points = np.float32([
    [0, 0],
    [600, 0],
    [600, 400],
    [0, 400]
])

# Calculate transformation matrix
matrix = cv2.getPerspectiveTransform(src_points, dst_points)

# Apply geometric correction
corrected = cv2.warpPerspective(img, matrix, (600, 400))

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(corrected, cv2.COLOR_BGR2RGB))
plt.title("Geometrically Corrected Image")
```

```
plt.axis("off")

plt.show()

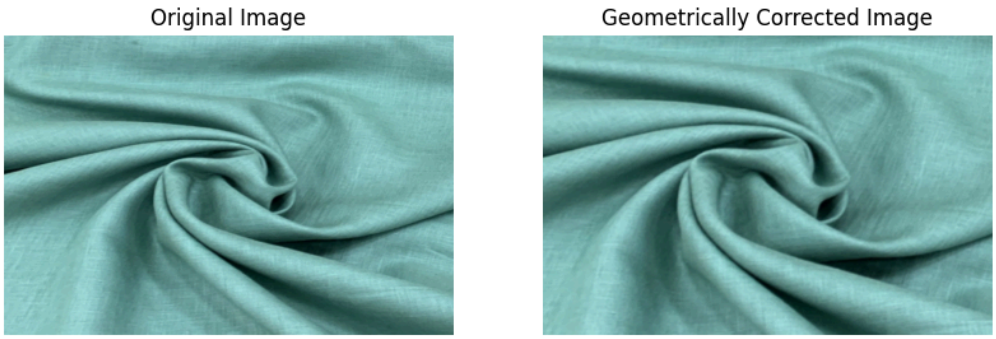
fabric_width_mm = 1000
image_width_pixels = 2040

mm_per_pixel = fabric_width_mm / image_width_pixels

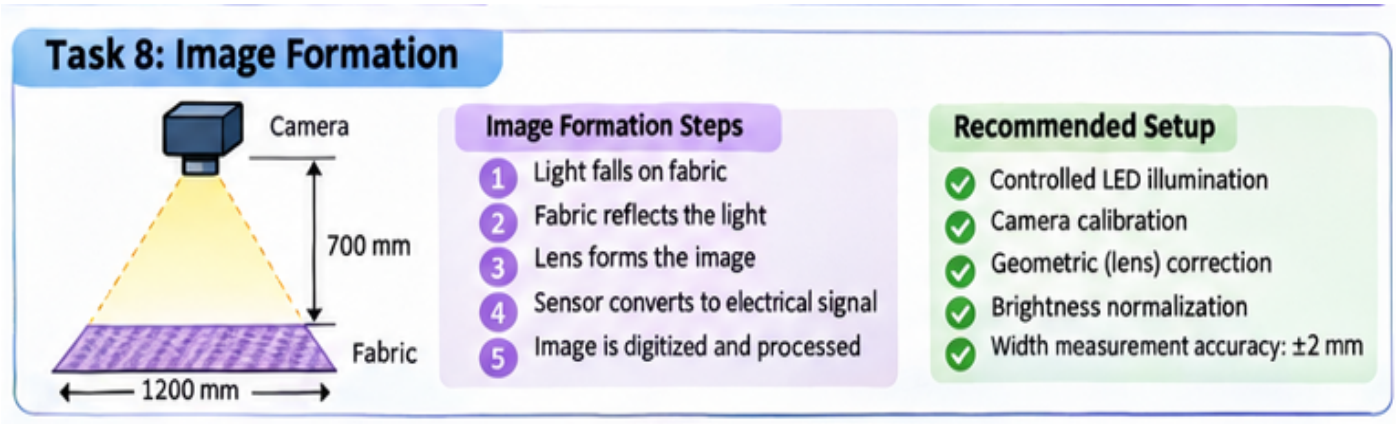
measured_width_pixels = 2040
measured_width_mm = measured_width_pixels * mm_per_pixel

print("Pixel Scale:", round(mm_per_pixel, 4), "mm/pixel")
print("Measured Fabric Width:", measured_width_mm, "mm")
```

```
print("Pixel Scale:", round(mm_per_pixel, 4), "mm/pixel")
print("Measured Fabric Width:", measured_width_mm, "mm")
```




```
Pixel Scale: 0.4902 mm/pixel
Measured Fabric Width: 1000.0 mm
```



TASK 9: Implementation and Performance Evaluation

Marks: 6

Design and compare three complete implementation configurations:

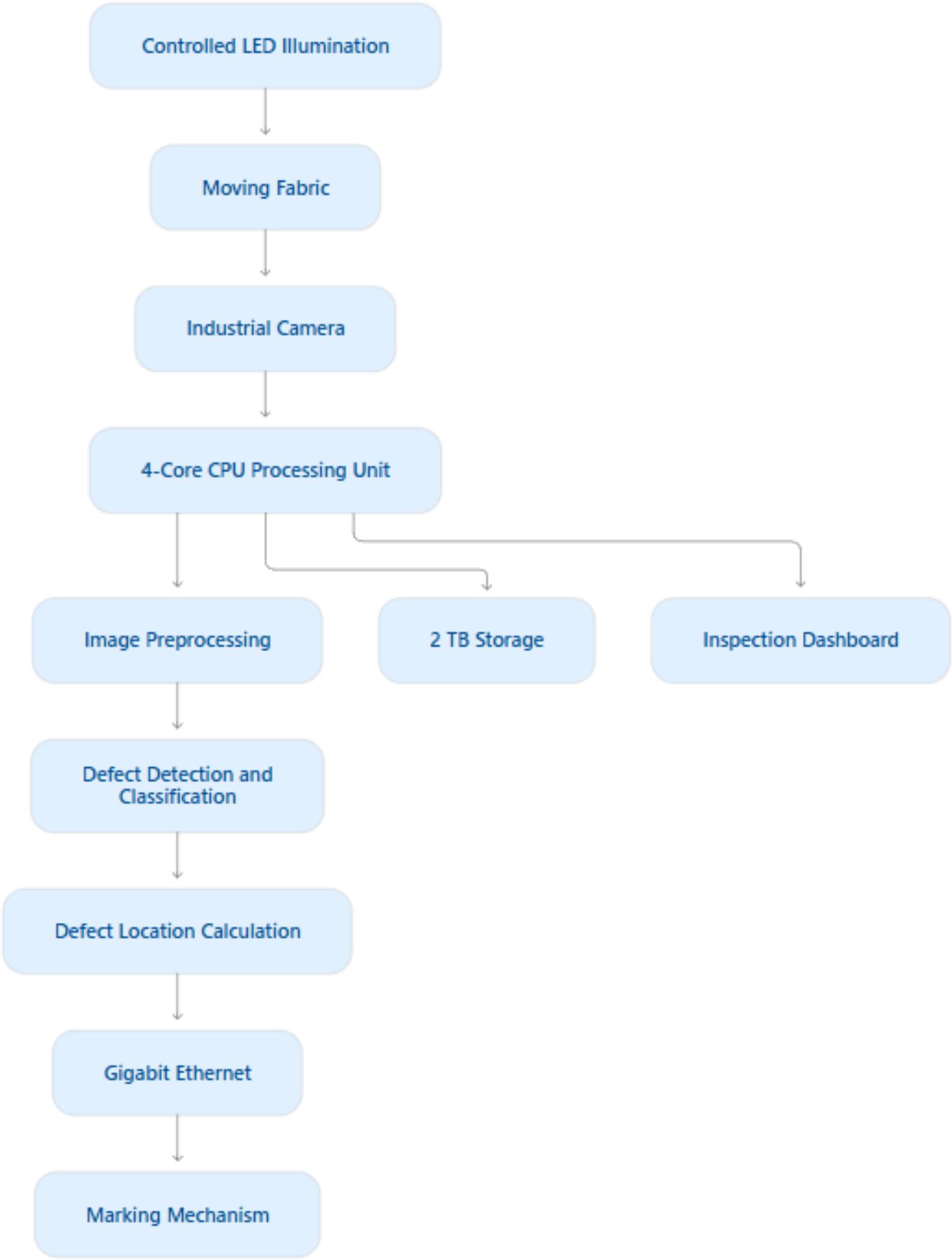
Candidate techniques / approaches:

- Budget configuration: minimum-cost camera, basic illumination and CPU-based processing
- Balanced configuration: suitable camera, controlled illumination and optimised CPU-based processing
- Premium configuration: high-resolution industrial camera, advanced illumination and higher-performance processing hardware

Student tasks:

44. Prepare a block diagram showing camera, illumination, processing unit, storage, network and marking mechanism.
45. Prepare a comparison table covering estimated cost, resolution, processing time, storage and expected inspection capability.
46. Explain how the system will detect and record holes, broken threads, stains and weaving irregularities.
47. Describe how the system will communicate the defect location to the marking mechanism located 0.3 m downstream.
48. Evaluate whether each configuration can meet the 0.5-second decision time, 7-day image retention and 2% maximum missed-defect rate.
49. Select one configuration that fits the ₹1,50,000 budget and justify your selection. Clearly state any assumptions used in cost estimates.

9.1 System Block Diagram


Controlled LED Illumination
Moving Fabric
Industrial Camera
4-Core CPU Processing Unit
Image Preprocessing
2 TB Storage
Inspection Dashboard
Defect Detection and Classification
Defect Location Calculation
Gigabit Ethernet
Marking Mechanism

9.2 Configuration Comparison

The following costs are approximate planning estimates, not vendor quotations. The exact performance depends on camera specifications, image size, illumination, and software optimization.

Parameter	Budget	Balanced	Premium
Camera	Basic industrial camera	Industrial global/line-scan camera	High-resolution line-scan camera
Resolution	1280 × 1024	2448 × 2048	4096 × 3000
Illumination	Basic LED	Controlled diffuse LED	Advanced multi-angle lighting
Processing	CPU	Optimized CPU	High-performance CPU/GPU
Estimated cost	₹60,000	₹1,40,000	₹2,50,000+
Processing time	0.3–0.5 s	0.2–0.4 s	0.1–0.3 s
Defect detection	Limited	Good	Excellent potential

The premium configuration exceeds the project budget and may require additional hardware.

9.3 Defect Detection Process

The system uses the following steps:

Capture fabric images.

Convert images into suitable colour representations.

Apply noise reduction.

Enhance image contrast.

Detect defect regions.

Extract features such as area, shape, colour, and texture.

Classify the defect.

Calculate the defect coordinates.

Record the defect information.

Send the marking signal.

9.4 Defect Classification

Defect	**Detection Technique**
Holes	Thresholding and contour detection
Broken threads	Edge and texture analysis
Oil stains	Contrast enhancement and colour analysis
Weaving irregularities	Texture feature analysis
Colour variations	RGB and HSV comparison

9.5 Marking Mechanism

The marking mechanism is located 0.3 m downstream from the camera.

Conveyor speed:

$$v = 0.6 \text{ m/s}$$

Distance:

$$d = 0.3 \text{ m}$$

Travel time:

$$t = \frac{d}{v}$$

$$t = \frac{0.3}{0.6} = 0.5 \text{ s}$$

Therefore, the fabric takes 0.5 seconds to travel from the camera to the marking mechanism.

The defect location is calculated using:

\[y_{\text{mark}}=y_{\text{camera}}+v\Delta t \]

The system should account for processing delay, communication delay, and actuator response time.

9.6 Performance Evaluation

Requirement	**Budget**	**Balanced**	**Premium**
0.5-second decision	Possible	Suitable	Suitable
7-day retention	Possible	Possible	Possible
2% missed-defect limit	Difficult to guarantee	Must be validated	Higher potential
₹1,50,000 budget	Fits	Fits with careful planning	Exceeds budget

Storage Calculation

Assume:

Medium-resolution grayscale images.

Image size = 2448 × 2048.

8-bit image.

1 image per second.

7 days of storage.

Image size:

$$2448 \times 2048 = 5,013,504 \text{ bytes}$$

Storage for 7 days:

$$5,013,504 \times 60 \times 60 \times 24 \times 7$$

$$= 3,032,164,? \text{ bytes}$$

Approximately:

$$\boxed{3.03 \text{ TB}}$$

This assumes one image per second continuously. It exceeds the available 2 TB storage.

Therefore, the system should use selective image retention, compression, or a lower capture frequency for archived images.

Note: The calculation is based on one grayscale image per second. Actual storage depends on capture rate and compression.

9.7 Recommended Configuration

Selected: Balanced Configuration

Reasons:

It fits the ₹1,50,000 budget with careful component selection.

It provides suitable image quality.

It can operate on CPU-based processing.

It supports controlled illumination.

It can provide accurate defect localization.

It offers a practical balance between cost, speed, and detection capability.

In conclusion, the implementation of a fabric inspection system requires the integration of image acquisition, preprocessing, defect detection, measurement, and reporting. The camera captures fabric images, and preprocessing improves their quality. Image processing or machine learning techniques are then used to identify defects. The system can classify defects and estimate their size, position, and severity. A dashboard can display the inspection results and help operators understand the fabric quality. Storage requirements depend on image resolution, bit depth, and the number of images collected. Different hardware configurations can be compared based on speed, accuracy, and cost. The system should be tested using representative fabric images before industrial deployment. The proposed architecture can reduce manual effort and improve inspection consistency. Therefore, integrating these components provides a practical approach to automated textile quality control.

Code:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Preprocessing
blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
# Thresholding
_, thresh = cv2.threshold(
    blur, 120, 255, cv2.THRESH_BINARY_INV
)

# Find defect contours
contours, _ = cv2.findContours(
    thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

# Draw defect locations
output = img.copy()

for contour in contours:
    area = cv2.contourArea(contour)

    if area > 50:
        x, y, w, h = cv2.boundingRect(contour)

        cv2.rectangle(
            output, (x, y), (x+w, y+h),
            (0, 255, 0), 2
        )
```

```
        cv2.putText(
            output, "Defect", (x, y-5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5,
            (0, 255, 0), 1
        )

plt.figure(figsize=(8, 5))
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
plt.title("Detected Defect Regions")
plt.axis("off")
plt.show()
start = time.time()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 50, 150)

end = time.time()

processing_time = end - start

print("Processing Time:", round(processing_time, 4), "seconds")

if processing_time <= 0.5:
    print("Decision Time Requirement: Satisfied")
```

```
else:
    print("Decision Time Requirement: Not Satisfied")width = 2448
height = 2048
bytes_per_pixel = 1

images_per_second = 1
days = 7

image_size = width * height * bytes_per_pixel

total_images = images_per_second * 60 * 60 * 24 * days

total_storage = image_size * total_images

storage_gb = total_storage / (1024 ** 3)

print("Image Size:", round(image_size / (1024 ** 2), 2), "MiB")
print("Total Images:", total_images)
print("Storage Required:", round(storage_gb, 2), "GiB")width = 2448
height = 2048
bytes_per_pixel = 1

images_per_second = 1
days = 7
```

image_size = width * height * bytes_per_pixel

total_images = images_per_second * 60 * 60 * 24 * days

total_storage = image_size * total_images

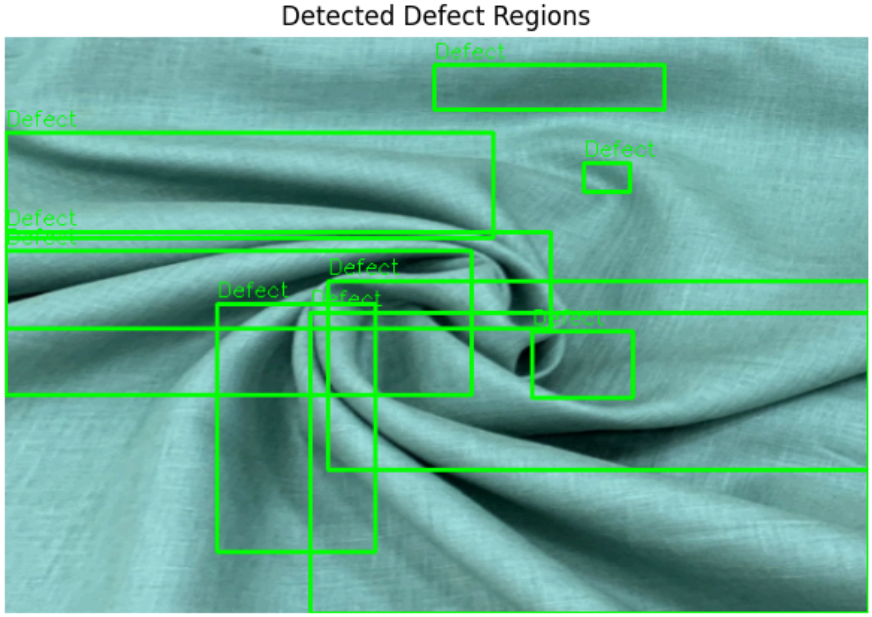
storage_gb = total_storage / (1024 ** 3)

print("Image Size:", round(image_size / (1024 ** 2), 2), "MiB")

print("Total Images:", total_images)

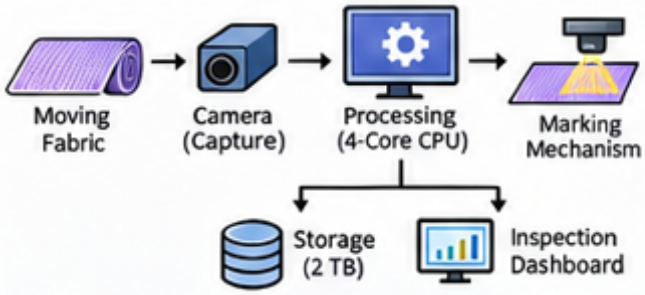
print("Storage Required:", round(storage_gb, 2), "GiB")

```
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
plt.title("Detected Defect Regions")
plt.axis("off")
plt.show()
```



Task 9: Implementation and Performance Evaluation

System Block Diagram



Configuration Comparison

Parameter	Budget	Balanced	Premium
Resolution	1280 × 1024	2448 × 2048	4096 × 3000
Est. Cost	₹ 60,000	₹ 1,40,000	₹ 2,50,000+
Processing Time	0.3 – 0.5 s	0.2 – 0.4 s	0.1 – 0.3 s
Detection Quality	Limited	Good	Excellent

Recommended: Balanced Configuration
(Fits ₹1,50,000 and meets the requirements)

TASK 10: Front-End Adaptation Challenge

Marks: 8

Assume that the textile company receives a new export order with the following changed requirements: the system must detect defects as small as 0.4 mm, identify stains visible under ultraviolet (UV) illumination, operate at a conveyor speed of 1.2 m/s, inspect twice the current production volume and produce a decision within 0.3 seconds. The existing PC and project budget must remain unchanged.

Candidate techniques / approaches:

- Retain the existing system with only software changes
- Modify the existing system by upgrading selected components
- Redesign the inspection system using a different acquisition and processing architecture

Student tasks:

50. Identify which existing system components will become inadequate under the new requirements.
51. Analyse the effect of the increased conveyor speed and smaller defect size on image sampling and acquisition.
52. Explain how UV illumination or an additional imaging channel could help identify the new stain type.
53. Compare the three adaptation strategies in terms of feasibility, accuracy, processing time, cost and implementation risk.
54. Recommend the most practical strategy under the unchanged PC and budget constraints.
55. Prepare an updated system block diagram and list the changes required.
56. Clearly identify any requirements that cannot realistically be achieved without additional resources, and justify your conclusion.

10.1 Inadequate Components

The following components may become inadequate:

Existing camera resolution.

Existing illumination system.

Current image-processing algorithm.

Camera acquisition rate.

CPU processing capacity.

Storage capacity.

Marking mechanism synchronization.

10.2 Effect of Increased Conveyor Speed

Existing speed:

$$v_1 = 0.6 \text{ m/s}$$

New speed:

$$v_2 = 1.2 \text{ m/s}$$

Speed increase:

$$\frac{1.2}{0.6} = 2$$

The fabric moves twice as fast.

Therefore:

Motion blur may increase.

More images must be processed per second.

The system requires faster acquisition.

Marking synchronization becomes more difficult.

10.3 Sampling Analysis for 0.4 mm Defects

For a defect to be represented by at least 5 pixels:

$$\text{Required Sampling Interval} = \frac{0.4}{5}$$

$$= 0.08 \text{ mm/pixel}$$

For a 1200 mm field of view:

$$\text{Required Image Width} = \frac{1200}{0.08}$$

$$\boxed{15,000 \text{ pixels}}$$

Therefore, a camera with approximately 15,000 pixels across the fabric width is required to represent a

0.4 mm defect with 5 pixels.

This is much higher than the existing medium-resolution camera.

10.4 UV Illumination

UV illumination can help identify stains that are not clearly visible under normal light.

Working principle:

UV light is directed onto the fabric.

Certain stains absorb or reflect UV differently.

Some materials produce fluorescence.

The camera captures the resulting image.

The processing system identifies the stain.

An additional UV imaging channel may improve stain detection.

However, the camera sensor and lens must be suitable for the selected UV wavelength.

10.5 Comparison of Adaptation Strategies

Parameter	Software Changes	Component Upgrades	Full Redesign
Cost	Low	Medium	High
Accuracy	Limited improvement	Good improvement	Highest potential
Processing time	Moderate improvement	Good improvement	Can be optimized
Implementation risk	Low	Medium	High
Feasibility	High	Moderate	Low under fixed budget

10.6 Recommended Strategy

Selected: Modify the Existing System by Upgrading Selected Components

Recommended modifications:

Upgrade the camera to a higher-resolution line-scan camera.

Add UV illumination.

Improve camera synchronization.

Use optimized image-processing algorithms.

Apply region-of-interest processing.

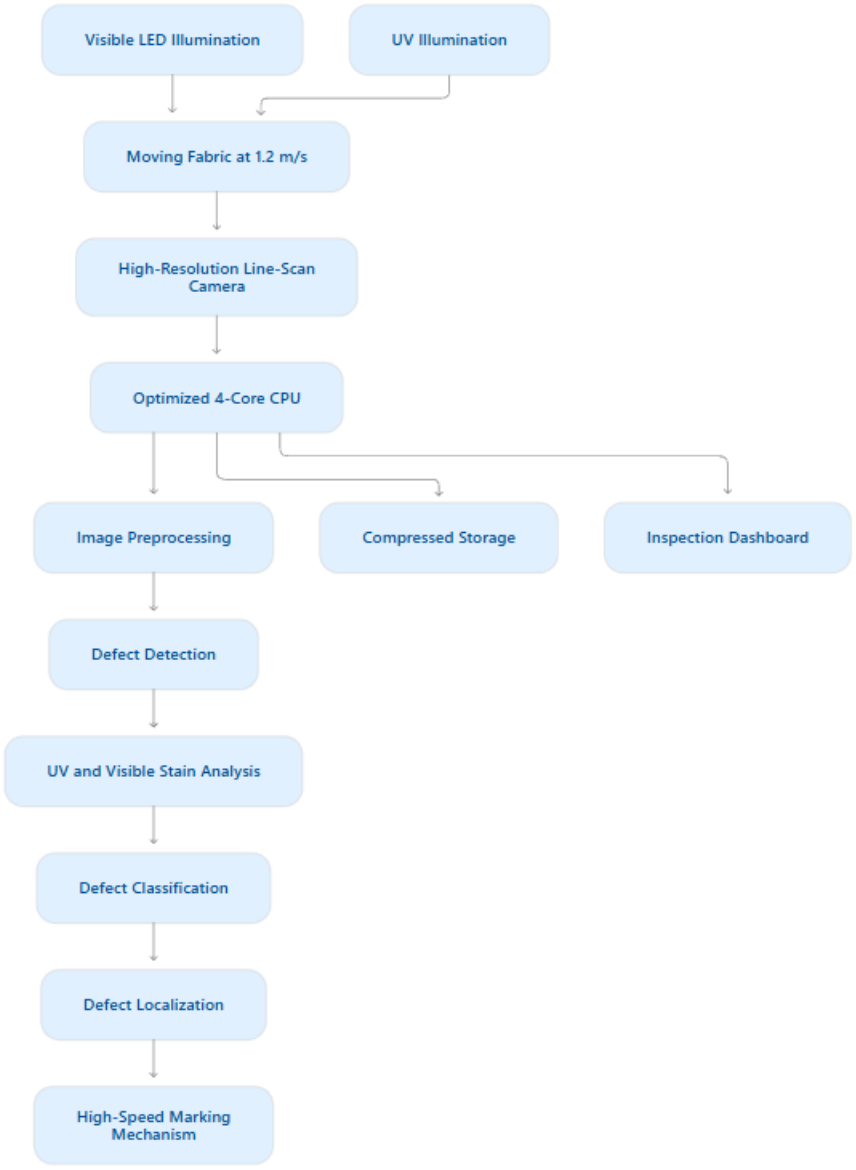
Use image compression for storage.

Optimize the marking mechanism.

Use parallel CPU processing where possible.

This approach is more practical than completely redesigning the system because the existing PC and budget cannot be increased.

10.7 Updated System Block Diagram



10.8 Requirements That Cannot Be Realistically Guaranteed

The following requirements cannot be guaranteed without additional resources or experimental validation:

1. Detection of 0.4 mm defects

The existing camera may not provide enough pixels across the defect.

2. Decision time of 0.3 seconds

The 4-core CPU may not process high-resolution images at the required speed.

3. Twice the production volume

The acquisition and processing rates must increase.

4. UV stain detection

The existing camera may not be suitable for UV imaging.

5. Missed-defect rate below 2%

This depends on the quality of the dataset, camera resolution, illumination, and detection algorithm.

Conclusion: Selected-component upgrades are the most practical approach. However, achieving all the new requirements simultaneously under the unchanged budget and PC is not realistically guaranteed. Additional resources may be necessary.

In conclusion, detecting small fabric defects at high conveyor speeds requires careful system design. A defect size of 0.4 mm creates a strict requirement for image resolution. The camera must capture enough pixels across the fabric to represent such defects clearly. The conveyor speed of 1.2 m/s also requires fast image acquisition and processing. The system must complete its decision within the specified 0.3-second time limit. UV imaging can be useful when certain defects show different responses under ultraviolet illumination. However, the suitability of UV detection depends on the fabric material and defect characteristics. High-resolution cameras, suitable lighting, and optimized algorithms are required to meet these conditions. The system should be evaluated using real fabric samples and measured performance results. Therefore, combining appropriate imaging techniques with efficient processing can support reliable high-speed fabric inspection.

Code:

```
fabric_width = 1200      # mm
defect_size = 0.4        # mm
```

```
required_pixels = 5      # Pixels across defect

sampling_interval = defect_size / required_pixels

required_resolution = fabric_width / sampling_interval

print("Required Sampling Interval:",
      sampling_interval, "mm/pixel")

print("Required Image Width:",
      int(required_resolution), "pixels")existing_speed = 0.6
new_speed = 1.2

existing_defect = 1.5
new_defect = 0.4

existing_decision_time = 0.5
new_decision_time = 0.3

print("Speed Increase:",
      new_speed / existing_speed, "times")

print("Defect Size Reduction:",
      existing_defect / new_defect, "times")
```

```
print("Decision Time Reduction:",
      existing_decision_time / new_decision_time, "times")# Simulate an additional UV channel
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Enhance contrast
uv_simulation = cv2.equalizeHist(gray)

# Detect bright regions
_, uv_mask = cv2.threshold(
    uv_simulation, 200, 255, cv2.THRESH_BINARY
)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(uv_simulation, cmap="gray")
plt.title("UV Channel Simulation")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(uv_mask, cmap="gray")
plt.title("Potential Stain Regions")
plt.axis("off")

plt.show()strategies = {
```

```
    "Software Changes": {
        "Cost": "Low",
        "Accuracy Improvement": "Limited",
        "Risk": "Low"
    },

    "Component Upgrades": {
        "Cost": "Medium",
        "Accuracy Improvement": "Good",
        "Risk": "Medium"
    },

    "Full Redesign": {
        "Cost": "High",
        "Accuracy Improvement": "High Potential",
        "Risk": "High"
    }
}

df = pd.DataFrame(strategies).T

print(df)
```

```
[13]
✓ 0s
```

```
    "Full Redesign": {
        "Cost": "High",
        "Accuracy Improvement": "High Potential",
        "Risk": "High"
    }
}

df = pd.DataFrame(strategies).T

print(df)
```

```
Required Sampling Interval: 0.08 mm/pixel
Required Image Width: 15000 pixels
Speed Increase: 2.0 times
Defect Size Reduction: 3.75 times
Decision Time Reduction: 1.6666666666666667 times
```

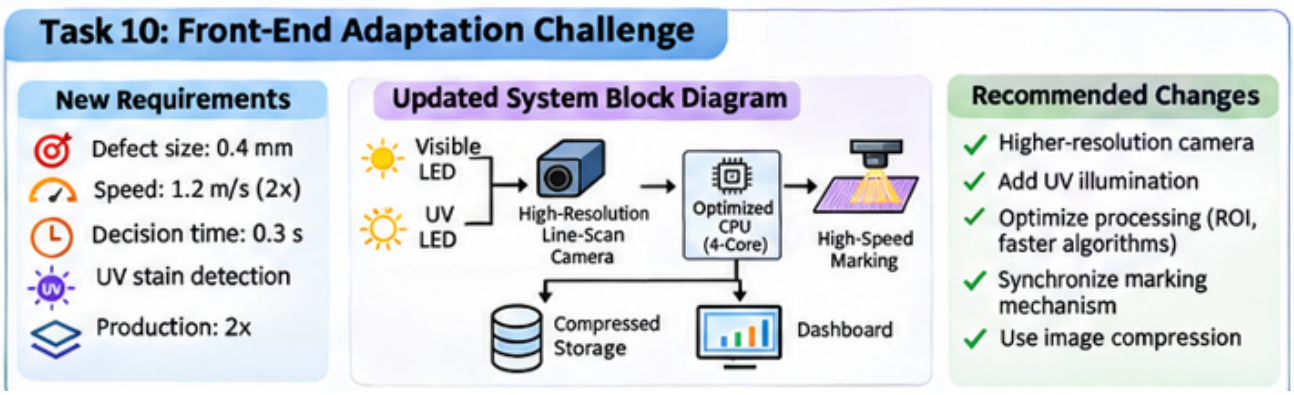
UV Channel Simulation



Potential Stain Regions



```
                      Cost Accuracy Improvement    Risk
Software Changes       Low              Limited     Low
Component Upgrades  Medium                 Good  Medium
Full Redesign         High       High Potential    High
```



Mark Distribution

Task	Topic	Marks
1	Overview and Features of Computer Vision	4
2	Levels of Computer Vision	5
3	Image Representation	4
4	Connectivity Analysis	4
5	Image Acquisition	5
6	Sampling Analysis	5
7	Quantization Analysis	4
8	Image Formation	5
9	Implementation and Performance Evaluation	6
10	Front-End Adaptation Challenge	8
	TOTAL	50

Submission Requirements

- Submit a neatly formatted assignment report with task-wise headings.
- Include hand-drawn or computer-generated diagrams wherever required.
- Show all numerical calculations with formulae, substitution and final units.
- Include comparison tables and evidence supporting the selected techniques.
- For implementation tasks, attach source code, sample input images and output screenshots.
- Mention references, tools and software libraries used.